

SinghArpit2492 Chess Project

Web Platform — Project Documentation

SinghArpit2492 Chess Engine & Web Platform

Comprehensive Project Documentation

This document provides an in-depth, A-to-Z explanation of the SinghArpit2492 Chess Project, detailing its architecture, directory structure, and the exact role of every file and codebase component.

1. Project Overview

The project is a modern, high-performance web-based chess application built with **React**, **TypeScript**, and **Vite**. It features a custom-built chess engine running in a Web Worker for unblocked UI performance, and a stunning, Lichess-inspired user interface with multiple themes and an interactive evaluation dashboard.

Key Technologies:

- **Frontend Framework:** React 18
 - **Language:** TypeScript (Strict Mode)
 - **Bundler:** Vite
 - **Routing:** React Router DOM
 - **Engine Concurrency:** HTML5 Web Workers
 - **Styling:** Vanilla CSS with extensive CSS variables for theming
-

2. Directory Structure Overview

The project resides in the [ChessWeb](#) directory and is organized logically into several subdirectories under `src/`:

```

ChessWeb/
  ■■■ public/           # Static assets
  ■■■ src/              # Main source code
  ■   ■■■ components/   # Reusable React UI components
  ■   ■■■ engine/       # The custom chess engine (Logic & AI)
  ■   ■■■ pages/        # Full-page route components
  ■   ■■■ styles/       # Global and modular CSS stylesheets
  ■   ■■■ App.tsx       # Main routing configuration
  ■   ■■■ main.tsx      # React application entry point
  ■   ■■■ types.ts      # Shared TypeScript interfaces and enums
  ■■■ index.html        # HTML entry point and metadata
  ■■■ package.json      # Project dependencies and scripts
  ■■■ tsconfig.json     # TypeScript configuration
  ■■■ vite.config.ts    # Vite bundler configuration

```

3. The Custom Chess Engine (src/engine/)

This directory contains the entire logic for the SinghArpit2492 Chess Engine. It is completely independent of the UI and processes everything using bitwise operations, 1D arrays, and highly optimized algorithms.

board.ts

Purpose: Manages the internal representation of the chess board.

- **Details:** It uses a 64-element 1D array to represent the squares. It keeps track of whose turn it is (`whiteToMove`), castling rights (using a bitmask), the en passant square, and the halfmove clock for the 50-move rule. It also maintains a history stack to allow undoing moves, and tracks the exact positions of both kings for fast check detection.

movegen.ts

Purpose: The Move Generator.

- **Details:** This file is responsible for calculating all legally possible moves for a given board state. It understands how every piece moves (using directional offsets for sliding pieces and knights). It handles complex chess rules including castling (checking if squares are attacked), en passant, and pawn promotion. It separates pseudo-legal moves from strictly legal moves (moves that don't leave the king in check).

eval.ts

Purpose: The Evaluation Function.

- **Details:** This is the "intuition" of the engine. It calculates a numerical score for the current board state. A positive score means White is winning; a negative score means Black is winning.
- **Material Weight:** It counts the value of pieces (Pawn=100, Knight=300, Bishop=300, Rook=500, Queen=900).

- **Positional Weight:** It uses Piece-Square Tables (PSTs) to encourage pieces to move to better squares (e.g., knights to the center, pawns advancing).

search.ts

Purpose: The Search Algorithm (Minimax with Alpha-Beta Pruning).

- **Details:** This file dictates how the engine "thinks ahead." It uses Iterative Deepening to search increasingly deeper into the move tree until the allocated time runs out. It employs Alpha-Beta pruning to discard bad branches of the move tree early, drastically speeding up computation. It returns the best move found, the evaluation score, the depth reached, and the number of nodes evaluated.

engine.worker.ts

Purpose: Web Worker Interface.

- **Details:** Chess engines require intense computation that would normally freeze a web browser. This file wraps the engine in a Web Worker, allowing it to run on a separate background thread. It communicates with the UI thread via `postMessage`, receiving the board state and time limit, and sending back the best move and analysis statistics once computation is complete.
-

4. User Interface Components (src/components/)

This folder holds the modular, reusable React components that make up the visual application.

ChessBoard.tsx

Purpose: Renders the 8x8 interactive chess board.

- **Details:** It takes the 64-square array and renders individual squares. It handles piece rendering based on the active theme (SVG, Classic Unicode, Letters, ASCII). It manages click events, highlighting the selected square, showing legal move target indicators (dots), and highlighting the last move played.

PlayerCard.tsx

Purpose: Displays player information above and below the board.

- **Details:** It shows the player's name, an avatar (Bot or Human), and a thinking indicator if the engine is calculating. It dynamically calculates and displays captured pieces and the material advantage (e.g., "+3").

EvalBar.tsx

Purpose: The vertical evaluation bar next to the board.

- **Details:** Visually represents the engine's evaluation score (`score`). It fills with white or black depending on who is winning, mathematically converting the centipawn score into a percentage for the CSS height property. It also displays "M" (Mate) if a forced checkmate is detected.

AnalysisPanel.tsx

Purpose: Displays engine statistics to the user.

- **Details:** A grid dashboard showing the current Evaluation, Search Depth, Total Nodes Evaluated, and Nodes Per Second (NPS).

MoveHistory.tsx

Purpose: The transcript of the game.

- **Details:** Displays a scrollable table of all moves played in standard algebraic notation (e.g., "Nf3", "e5"), arranged in White/Black pairs.

SidebarControls.tsx

Purpose: The settings and configuration panel.

- **Details:** Contains interactive buttons and sliders to allow the user to change the Piece Style, Board Theme, Engine Think Time, and start a new game playing as White or Black.

TabPanel.tsx

Purpose: A layout component for the sidebar.

- **Details:** Manages the tabbed interface (Moves, Settings, Analysis), allowing the user to switch between different sidebar views smoothly.
-

5. Application Pages (src/pages/)

LandingPage.tsx

Purpose: The welcome screen (/).

- **Details:** A cinematic, aesthetically pleasing entry point. It features the "SinghArpit2492" branding, animated background elements, floating chess pieces, and feature cards explaining the project's capabilities. A "Play Now" button routes the user to the actual game.

GamePage.tsx

Purpose: The main application interface (/play).

- **Details:** This is the core orchestrator. It holds the React State for the board, move history, active themes, and game over status. It spawns the Web Worker (`engine.worker.ts`) and handles the logic for human moves vs. engine moves. It arranges all the smaller components (`ChessBoard`, `Sidebar`, `EvalBar`) into a cohesive, responsive Lichess-style layout using CSS Grid and Flexbox.
-

6. Styles & Theming (src/styles/)

The project uses Vanilla CSS powered heavily by CSS Variables for instantaneous theme switching.

index.css

- Contains global CSS variables (`:root`), declaring colors, typography (Inter & JetBrains Mono), spacing, and animations. It includes the logic for data-attribute theme switching (e.g., `[data-board-theme='cyberpunk']`).

board.css

- Specific styles for the `ChessBoard.tsx`. Handles the grid layout, square colors, target dots, hover effects, and the precise scaling and shadows for the various piece types.

game.css

- Styles the `GamePage.tsx` layout. Manages the responsive design, ensuring the board scales correctly on mobile, and styles the sidebar tabs, move history tables, and player cards.

landing.css

- Handles the complex animations (floating pieces, pulsating gradients, glowing buttons) required for the premium `LandingPage.tsx` aesthetic.

7. Configuration & Entry

App.tsx

- Sets up the `react-router-dom` `BrowserRouter`, directing visitors to `<LandingPage />` on the root route and `<GamePage />` on `/play`.

main.tsx

- The standard React DOM entry point that binds `<App />` to the `div#root` in the HTML document.

types.ts

- A central repository for TypeScript interfaces. It defines enums for Pieces (combining color and type via bitwise flags), the structure of a `Move`, the `EngineResult`, and the available theme literal types (`PieceTheme`, `BoardTheme`).

index.html & vite.config.ts

- `index.html` contains the SEO meta tags and the title "SinghArpit2492 Chess Engine".
 - `vite.config.ts` configures the build process and is set up to automatically open the browser when the dev server starts.
-

Conclusion

The SinghArpit2492 Chess Project is a complete, full-stack client-side application. By separating the computationally heavy chess engine into a Web Worker and utilizing React for a highly dynamic, themeable UI, it achieves both professional-grade performance and a premium user experience.